

STATIC WEBSITE HOSTING USING AMAZON S3

Prepared in the partial fulfilment of the Summer Internship Program on AWS

AT



Under the guidance of

Mrs BETHALA SUMANA, APSSDC

Mr JUTTUKA LOVABABU, APSSDC

Submitted by

BANDI MOHAN SAI MANIKANTA, A24126552009, ANITS, (Batch -1)

DEVARAPALLI CHARAN, A24126552018, ANITS, (Batch -1)

ADIMOLU V V NAGA DURGESH, A24126552003, ANITS, (Batch -1)

SADAPU SANJAY RISHI, A24126552052, ANITS, (Batch -1)

TANNI DHANUNJAY, A24126552059, ANITS, (Batch -1)

ABSTRACT

This project presents the design, implementation, and monitoring of a secure, highly available static website hosting solution on Amazon Web Services (AWS). Rather than exposing storage directly to the public internet, the architecture uses Amazon S3 purely as a private origin, with Amazon CloudFront acting as the sole public-facing entry point, providing global content delivery and free HTTPS encryption through its default domain certificate.

Access between CloudFront and S3 is restricted using Origin Access Control (OAC) and an auto-generated least-privilege bucket policy, ensuring the storage bucket itself is never publicly reachable. Deployment is fully automated using a GitHub Actions CI/CD workflow: every push to the repository's main branch triggers an automatic sync of website files to S3 and a CloudFront cache invalidation, so changes go live within seconds, without any manual console interaction.

Authentication between GitHub Actions and AWS is implemented using OpenID Connect (OIDC) federation through AWS IAM, eliminating the need for long-lived access keys entirely. A dedicated IAM role, scoped to only the specific S3 bucket and CloudFront distribution used in this project, is assumed temporarily on each workflow run.

To close the loop on operational visibility, Amazon CloudWatch monitors CloudFront's error rate in real time, and Amazon SNS delivers an email notification the moment that error rate breaches a defined threshold — giving the team an active alerting mechanism rather than a passive dashboard. Together, S3, CloudFront, IAM, CloudWatch, and SNS form the five core AWS services used in this project, supported by GitHub Actions as the CI/CD automation layer.

The result is a production-representative static hosting pipeline that is secure by default, cost-free at this traffic scale (running entirely within CloudFront's Free flat-rate plan and AWS's always-free service tiers), actively monitored, and requires zero manual deployment effort after initial setup. This report documents the architecture, build methodology, technology stack, implementation details, challenges encountered, and results of the project.

TABLE OF CONTENTS

1. INTRODUCTION.....	4
2. METHODOLOGY.....	5
2.1. Requirements & Planning.....	5
2.2. Storage Layer Setup.....	5
2.3. Content Delivery Setup.....	5
2.4. Access Control Setup.....	5
2.5. CI/CD Pipeline Setup.....	5
2.6. Monitoring & Alerting Setup.....	5
2.7. Verification.....	6
3. TECHNOLOGY STACK.....	7
4. SYSTEM DESIGN / ARCHITECTURE.....	8
4.1. Deployment Flow.....	8
4.2. Content Delivery Layer — Amazon CloudFront.....	9
4.3. Access Control Layer — S3 Bucket Policy.....	10
4.4. Identity Layer — IAM OIDC Federation.....	10
4.5. CI/CD Layer — GitHub Actions.....	11
4.6. Monitoring & Alerting Layer — CloudWatch and SNS.....	12
5. IMPLEMENTATION.....	14
5.1. S3 Bucket Configuration.....	14
5.2. CloudFront Distribution Configuration.....	14
5.3. IAM OIDC Role Configuration.....	14
5.4. GitHub Actions Workflow.....	14
5.5. CloudWatch Alarm & SNS Configuration.....	14
5.6. Cost & Billing Safeguards.....	15
6. RESULTS.....	16
6.1. Secure, Private Origin.....	16
6.2. HTTPS Delivery with No Custom Domain Cost.....	16
6.3. End-to-End Automated Deployment.....	16
6.4. Zero Stored Credentials.....	16
6.5. Verified Alerting Pipeline.....	16
6.6. Cost Outcome.....	16
6.7. Demonstration Materials.....	16
7. TEAM CONTRIBUTIONS.....	17
8. CHALLENGES & LEARNINGS.....	18
9. RESOURCE TEARDOWN.....	19
10. CONCLUSION.....	20
REFERENCES.....	21

1. INTRODUCTION

Hosting a static website — a portfolio, documentation site, or landing page consisting of HTML, CSS, JavaScript, and static assets — no longer requires a traditional web server. Cloud object storage combined with a content delivery network (CDN) can serve the same content faster, more securely, and at a fraction of the operational cost of a conventional server.

This project implements exactly that pattern on AWS. Amazon S3 provides durable, low-cost storage for the website's files. Amazon CloudFront sits in front of S3 as a global CDN, caching content at edge locations close to visitors and terminating HTTPS connections so that traffic between the visitor and AWS is always encrypted. Critically, the S3 bucket itself is kept completely private; the only entity permitted to read from it is the specific CloudFront distribution created for this project, enforced through Origin Access Control and a bucket policy scoped to that distribution's ARN.

A key objective of this project was to go beyond a one-time manual upload and instead build a repeatable deployment pipeline. GitHub Actions was integrated so that every code change pushed to the repository is automatically synchronized to S3 and the CloudFront cache is invalidated, giving the workflow of a real production deployment pipeline. Authentication for this pipeline uses IAM's OpenID Connect (OIDC) federation, a modern alternative to storing static AWS access keys as GitHub secrets.

A second objective, introduced after the initial deployment was verified, was to add operational monitoring: rather than only viewing metrics passively in a dashboard, the team configured a CloudWatch alarm on CloudFront's error rate that actively notifies the team by email through Amazon SNS the moment a problem is detected.

Finally, since this project was built for learning and portfolio purposes rather than sustained production traffic, cost control was treated as a first-class design constraint throughout — from choosing CloudFront's \$0/month Free flat-rate plan, to avoiding services with ongoing charges such as a purchased domain or Route 53 hosted zone, to planned teardown of all resources after review.

2. METHODOLOGY

The project was executed in sequential, checkpointed phases, allowing each layer of the architecture to be verified in the AWS console before the next layer was built on top of it.

2.1. Requirements & Planning

The primary goals were defined up front: a live HTTPS website, private backend storage, automated deployment on every code push, minimal ongoing cost, active error monitoring, and a stack that could be fully documented and torn down after review without losing the ability to demonstrate the work.

2.2. Storage Layer Setup

An Amazon S3 bucket was created with all Block Public Access settings enabled, and the website's HTML, CSS, and JavaScript files were uploaded. Static Website Hosting was deliberately left disabled, since that feature requires a public bucket and is incompatible with the CloudFront Origin Access Control pattern used in this project.

2.3. Content Delivery Setup

A CloudFront distribution was created with the S3 bucket as its origin, using Origin Access Control so that CloudFront — and only CloudFront — could read from the bucket. The default root object was configured as index.html, and the distribution was placed on CloudFront's Free flat-rate plan, which bundles CDN, WAF, DDoS protection, and a TLS certificate into a \$0/month tier suited to low-traffic personal projects.

2.4. Access Control Setup

An IAM OIDC identity provider trusting GitHub's token issuer (`token.actions.githubusercontent.com`) was registered in AWS. A dedicated IAM role was then created with a trust policy restricting it to this project's specific GitHub repository and branch, and an inline least-privilege policy granting only the S3 and CloudFront actions required for deployment.

2.5. CI/CD Pipeline Setup

A GitHub Actions workflow was added to the repository, triggered on every push to the default branch. The workflow authenticates to AWS via the OIDC role (no stored credentials), synchronizes the repository's website files to S3, and issues a CloudFront cache invalidation so updates are visible immediately rather than waiting for cached content to expire.

2.6. Monitoring & Alerting Setup

An Amazon SNS topic was created with an email subscription, and a CloudWatch alarm was configured to watch CloudFront's 5xxErrorRate metric. When the alarm transitions into the ALARM state, it publishes to the SNS topic, which delivers an email notification to the team within roughly a minute of the state change.

2.7. Verification

At each phase, configuration was verified directly in the AWS console — confirming the auto-generated bucket policy, distribution status, IAM trust relationship, a successful end-to-end GitHub Actions run, and a real SNS email delivery — before moving to the next phase.

3. TECHNOLOGY STACK

The project uses five AWS services alongside GitHub Actions for automation, chosen to keep the stack minimal and low-cost while still reflecting production-representative patterns.

Service	Role in this project
Amazon S3	Private object storage for website files (HTML, CSS, JS). Never exposed directly to the public internet.
Amazon CloudFront	Global CDN and sole public entry point. Provides free HTTPS via its default *.cloudfront.net certificate, edge caching, and DDoS protection under the Free flat-rate plan.
AWS IAM (OIDC)	Issues short-lived, scoped credentials to GitHub Actions via federated identity — no long-lived access keys stored anywhere.
Amazon CloudWatch	Continuously evaluates CloudFront's error-rate metric and triggers an alarm when it crosses a defined threshold.
Amazon SNS	Delivers a real-time email notification to the team the moment the CloudWatch alarm enters the ALARM state.
GitHub Actions	CI/CD automation — syncs files to S3 and invalidates the CloudFront cache on every push to the main branch. (Not an AWS service.)

Note: Amazon Route 53 and AWS Certificate Manager (ACM) were deliberately excluded from this build to avoid the recurring cost of a registered domain name. CloudFront's default domain already provides free, valid HTTPS, so no functionality was lost by omitting a custom domain.

4. SYSTEM DESIGN / ARCHITECTURE

The architecture follows a private-origin, CDN-fronted pattern: nothing reaches S3 directly, every request is served through CloudFront, every deployment is triggered by a GitHub push rather than a manual upload, and every error condition is actively reported rather than passively logged.

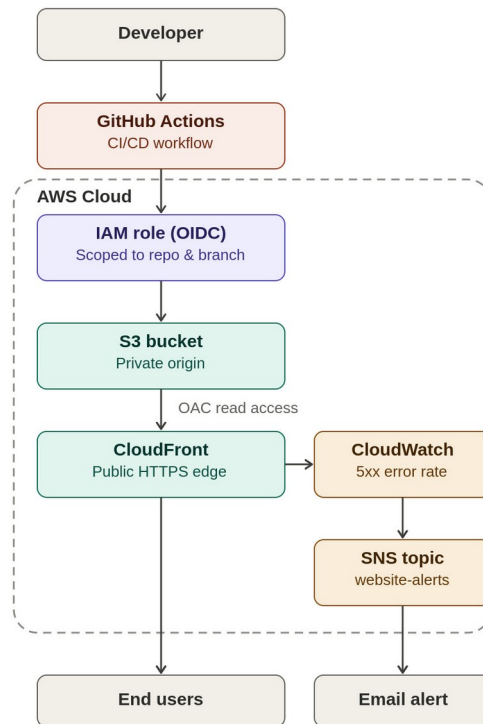


Figure 1. End-to-end architecture — deployment path (left) and monitoring/alerting path (right), both scoped within the AWS Cloud boundary.

4.1. Deployment Flow

1. A developer pushes code to the GitHub repository's main branch.
2. GitHub Actions authenticates to AWS by assuming the IAM OIDC role — no stored secrets are used.
3. The workflow runs "aws s3 sync" to upload changed files to the S3 bucket.
4. The workflow issues a CloudFront cache invalidation so the CDN immediately serves fresh content.
5. End users load the site over HTTPS via the CloudFront distribution domain, which fetches from S3 only on a cache miss.
6. CloudWatch continuously evaluates CloudFront's error rate in the background; if it breaches the configured threshold, SNS emails the team.

4.2. Content Delivery Layer — Amazon CloudFront

The distribution was created with the S3 bucket as its origin and Origin Access Control enabled, meaning CloudFront automatically writes and maintains the S3 bucket policy restricting access to itself alone.

Step 1 Choose a plan
Step 2 Get started
Step 3 Specify origin
Step 4 Enable security
Step 5 Review and create

Review and create

General configuration [Edit](#)
Distribution name: my-aws-pro-static-website-hosting
Description: -
Billing: Free (\$0/month)
Today's pro-rated charge (estimate): \$0

Origin [Edit](#)

Because you granted CloudFront access to your origin, CloudFront can write and update S3 bucket policies that restrict access to your S3 origin to CloudFront.

S3 origin: my-aws-pro-static-website-hosting.s3.us-east-1.amazonaws.com	Origin path: -	Grant CloudFront access to origin: Yes	Enable Origin Shield: No
Connection attempts: 3	Connection timeout: 10		

Cache settings [Edit](#)
CloudFront will apply default cache settings tailored to serving content from a S3 origin. You can customize settings after you create your distribution.

Security [Edit](#)
Security protections: Enabled
Use monitor mode: No

Usage and transfer limits will be prorated for your first month. Subject to [usage](#) and [transfer limits](#).

[Cancel](#) [Previous](#) [Create distribution](#)

Figure 2. Reviewing the CloudFront distribution configuration before creation.

my-aws-pro-static-website-hosting [Free plan](#) [View metrics](#)

Details
Distribution domain name: dsagewuhtg9.cloudfront.net
Billing: Free plan (\$0/month) [Manage plan](#)
ARN: arn:aws:cloudfront::736461511133:distribution/E9XVPXUZEEDFD
Last modified: Deploying

[General](#) [Security](#) [Origins](#) [Behaviors](#) [Error pages](#) [Invalidations](#) [Logging](#) [Tags](#)

Settings [Edit](#)
Name: my-aws-pro-static-website-hosting
Description: -
Price class: Use all edge locations (best performance)
Supported HTTP versions: HTTP/2, HTTP/1.1, HTTP/1.0
Alternate domain names: [Add domain](#)
Standard logging: Available with the Pro plan
Default root object: -

Figure 3. CloudFront distribution confirmed on the Free flat-rate plan (\$0/month).

my-aws-pro-static-website-hosting [Free plan](#) [View metrics](#)

Details
Distribution domain name: dsagewuhtg9.cloudfront.net
Billing: Free plan (\$0/month) [Manage plan](#)
ARN: arn:aws:cloudfront::736461511133:distribution/E9XVPXUZEEDFD
Last modified: July 12, 2026 at 7:58:42 AM UTC

[General](#) [Security](#) [Origins](#) [Behaviors](#) [Error pages](#) [Invalidations](#) [Logging](#) [Tags](#)

Origins (1) [Edit](#) [Delete](#) [Create origin](#)

Origin name	Origin domain	Origin path	Origin type	Origin Shield region	Origin access
my-aws-pro-static-website-hosting	my-aws-pro-static-website...		S3	-	E1RMUBYWQI29RO

Origin groups: High-availability origin failover
Keep your application running even if your primary origin fails. CloudFront automatically switches to your backup origin when your primary origin becomes unavailable or returns error responses, maintaining availability without disrupting users.

[Upgrade to Premium](#)

Figure 4. Origins configuration — S3 origin with Origin Access Control attached.

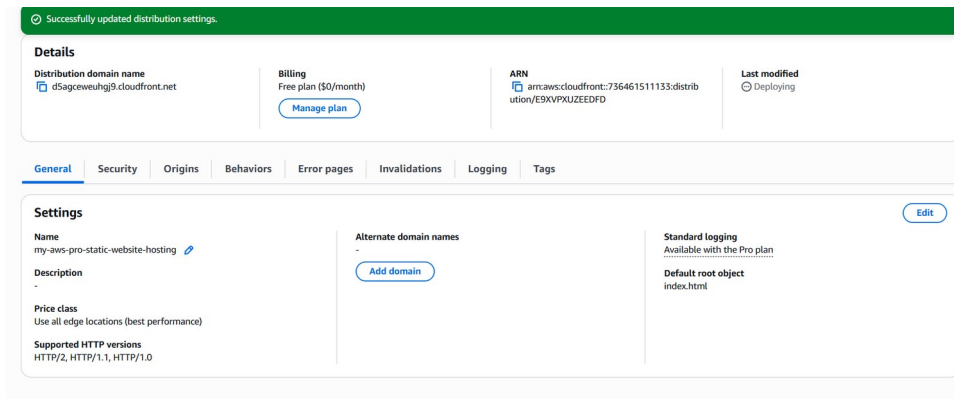


Figure 5. Default root object set to index.html after a distribution settings update.

4.3. Access Control Layer — S3 Bucket Policy

CloudFront's Origin Access Control automatically generated the following least-privilege bucket policy, restricting S3 read access to only this project's specific CloudFront distribution ARN:

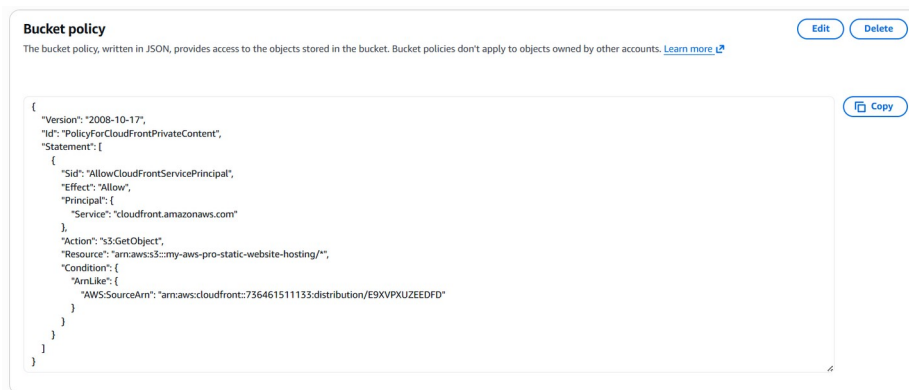


Figure 6. Auto-generated bucket policy scoping s3:GetObject to a single CloudFront distribution.

4.4. Identity Layer — IAM OIDC Federation

An IAM role was configured to trust GitHub's OIDC token issuer, with its trust policy restricted to this project's specific repository and branch — meaning only workflow runs from that exact repository can assume the role.

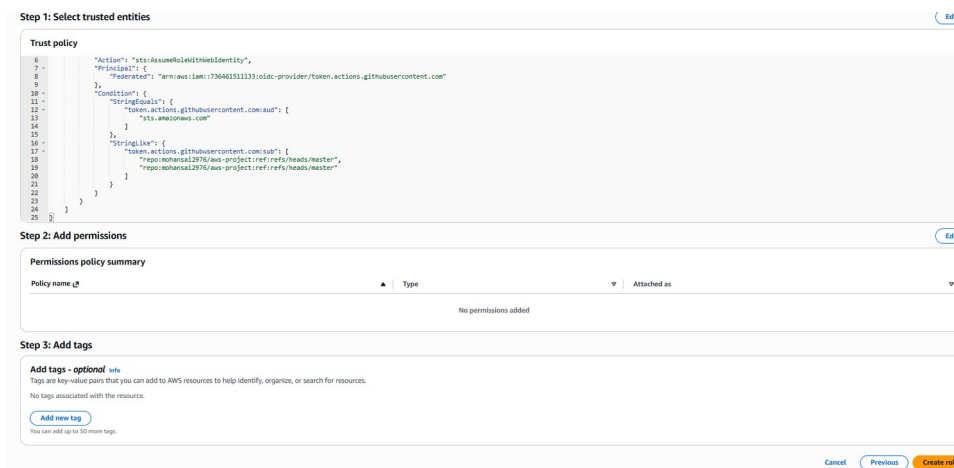


Figure 7. IAM role trust policy scoped to the project's GitHub repository and branch.

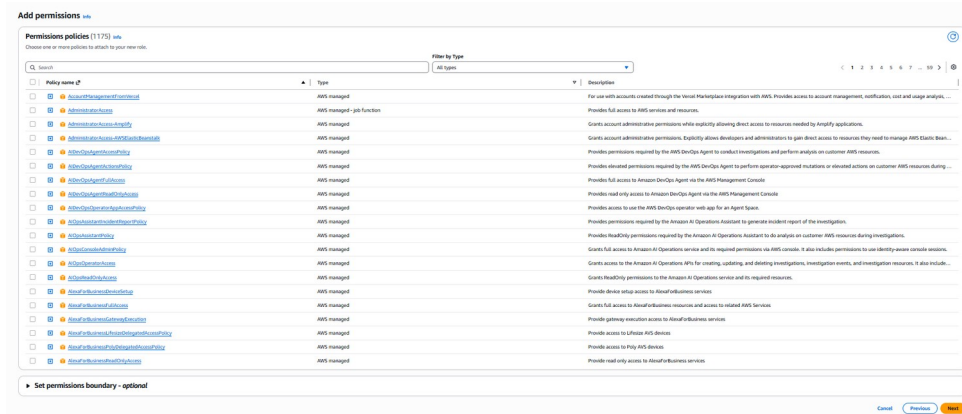


Figure 8. Searching AWS-managed policies while attaching permissions to the role.

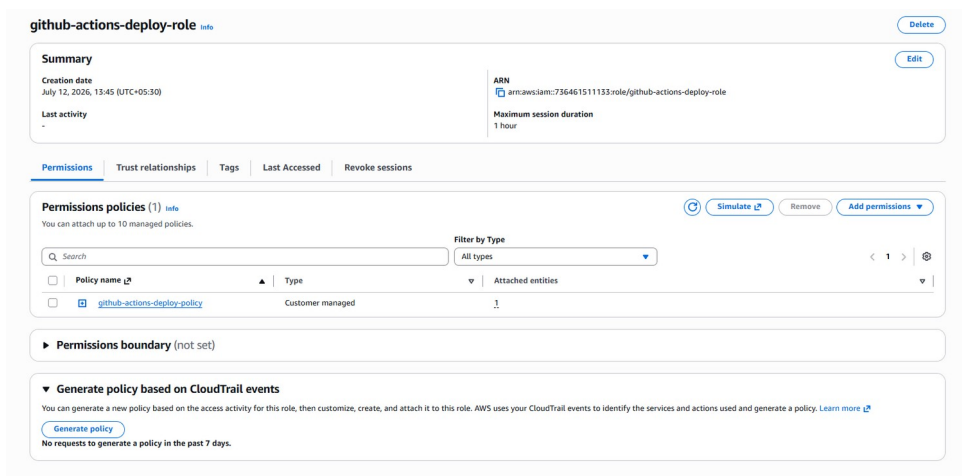


Figure 9. Completed IAM role with a least-privilege deployment policy attached.

4.5. CI/CD Layer — GitHub Actions

The workflow file below drives the entire deployment process on every push to the main branch:

```
name: Deploy to S3

on:
  push:
    branches: [master]

permissions:
  id-token: write
  contents: read

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::<account-id>:role/github-actions-deploy-role
          aws-region: us-east-1

      - name: Sync files to S3
```

```

    run: aws s3 sync . s3://<bucket-name> --delete --exclude ".git/*" --exclude
    ".github/*"

  - name: Invalidate CloudFront cache
    run: aws cloudfront create-invalidation --distribution-id <distribution-id>
    --paths "/*"

```

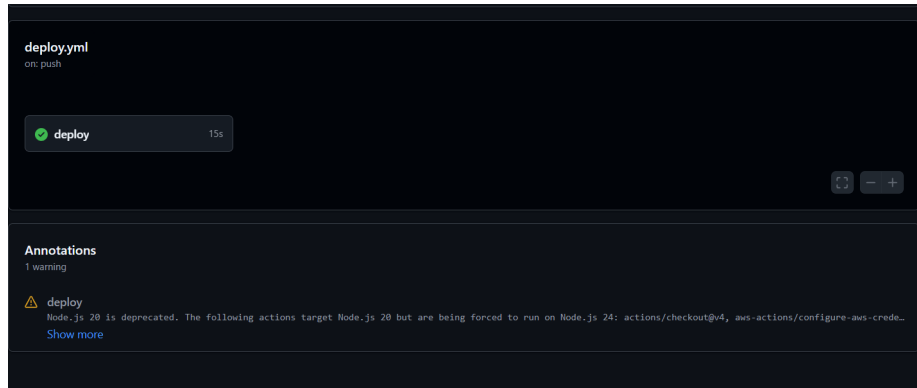


Figure 10. Successful end-to-end GitHub Actions deployment run.

4.6. Monitoring & Alerting Layer — CloudWatch and SNS

CloudFront automatically publishes request count, data transfer, error rate, and cache hit ratio metrics to CloudWatch with no additional configuration. Rather than relying solely on a passive dashboard, a CloudWatch alarm was created on the 5xxErrorRate metric — chosen because a 5xx response indicates a genuine infrastructure fault (for example, a broken bucket policy), unlike a 4xx response, which is often just a visitor mistyping a URL and is treated as normal background noise for a low-traffic static site.

☐ DistributionId 6/6	▼ Region	▼ Metric name	▼ Alarms
☐ E9XVPXUZEEDFD	Global	4xxErrorRate ⓘ	No alarms
☐ E9XVPXUZEEDFD	Global	Requests ⓘ	No alarms
☐ E9XVPXUZEEDFD	Global	TotalErrorRate ⓘ	No alarms
☐ E9XVPXUZEEDFD	Global	BytesUploaded ⓘ	No alarms
☐ E9XVPXUZEEDFD	Global	BytesDownloaded ⓘ	No alarms
☐ E9XVPXUZEEDFD	Global	5xxErrorRate ⓘ	No alarms

Figure 11. CloudFront metrics available in CloudWatch for the project's distribution, including 4xxErrorRate and 5xxErrorRate.

The alarm's notification action was attached directly to an Amazon SNS topic, so that an "In alarm" state transition triggers an email to the team automatically:

Configure actions

Notification

Alarm state trigger
Define the alarm state that will trigger this action.

☒ In alarm
The metric or expression is outside of the defined threshold.

☐ OK
The metric or expression is within the defined threshold.

☐ Insufficient data
The alarm has just started or not enough data is available.

Send a notification to the following SNS topic
Define the SNS Simple Notification Service topic that will receive the notification.

☒ Select an existing SNS topic

☐ Create new topic

☐ Use topic ARN to notify other accounts

Send a notification to...

Q Select an SNS topic

Only topics belonging to this account are listed here. All persons and applications subscribed to the selected topic will receive notifications.

Add notification

Lambda action

Add Lambda action

Auto Scaling action

Add Auto Scaling action

Figure 12. Attaching the SNS topic "website-alerts" as the alarm's notification action.

Because the project's real traffic volume is low, the alarm was verified using CloudWatch's built-in alarm testing feature, which forces a genuine OK → ALARM state transition — the same transition a real error spike would cause — without requiring the team to intentionally break the live site:

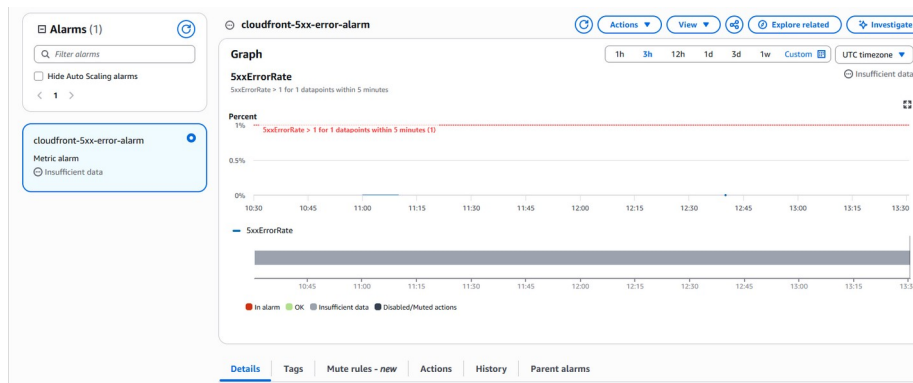


Figure 13. The 5xxErrorRate alarm graph, showing the configured 1% threshold over a 5-minute evaluation period.

ALARM: "cloudfront-5xx-error-alarm" in US East (N. Virginia) [Inbox](#)

AWS Notifications aws-notifications@amazonaws.com 7:09 PM (1 minute ago) ☆ ↻ ⓘ

You are receiving this email because your Amazon CloudWatch Alarm "cloudfront-5xx-error-alarm" in the US East (N. Virginia) region has entered the ALARM state, because "Manual test trigger for demo" at "Sunday 12 July, 2026 13:36:30 UTC".

View this alarm in the AWS Management Console:
<https://us-east-1.console.aws.amazon.com/cloudwatch/home?region=us-east-1#alarmsV2:alarm/cloudfront-5xx-error-alarm>

Alarm Details:

- Name: cloudfront-5xx-error-alarm
- Description:
- State Change: INSUFFICIENT_DATA → ALARM
- Reason for State Change: Manual test trigger for demo
- Timestamp: Sunday 12 July, 2026 13:36:30 UTC
- AWS Account: 736461511133
- Alarm Arn: arn:aws:cloudwatch:us-east-1:736461511133:alarm:cloudfront-5xx-error-alarm

Threshold:

- The alarm is in the ALARM state when the metric is GreaterThanThreshold 1.0 for at least 1 of the last 1 period(s) of 300 seconds.

Monitored Metric:

- MetricNamespace: AWS/CloudFront
- MetricName: 5xxErrorRate
- Dimensions: (Region = Global) [Distributed = EKVVPXUZEEDFD]
- Period: 300 seconds
- Statistic: Average
- Unit: not specified
- TreatMissingData: missing

State Change Actions:

- OK:
- ALARM: [arn:aws:sns:us-east-1:736461511133:website-alerts]
- INSUFFICIENT_DATA:

If you wish to stop receiving notifications from this topic, please click or visit the link below to unsubscribe:
<https://sns.us-east-1.amazonaws.com/subscribe.html?SubscriptionArn=arn:aws:sns:us-east-1:736461511133:website-alerts:5023eaf34034a5b-befb-a5d8aa48e9b8Endpoint=pa@viamer17@gmail.com>

Figure 14. SNS email alert received after the CloudWatch alarm entered the ALARM state — confirming the end-to-end monitoring pipeline.

5. IMPLEMENTATION

This section details how each layer of the design was translated into a working configuration.

5.1. S3 Bucket Configuration

- Bucket created with all four Block Public Access settings enabled
- Static Website Hosting left disabled (incompatible with the private-origin OAC pattern)
- Website files (index.html, error.html, CSS, JS) uploaded directly to the bucket root

5.2. CloudFront Distribution Configuration

- Origin: S3 bucket, connected via Origin Access Control (not the legacy S3 static-website endpoint)
- Viewer protocol policy: Redirect HTTP to HTTPS
- Default root object: index.html
- Pricing/plan: Free flat-rate plan (\$0/month), which also bundles WAF and DDoS protection

5.3. IAM OIDC Role Configuration

Inline least-privilege policy attached to the deployment role:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:PutObject",
        "s3:DeleteObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::<bucket-name>",
        "arn:aws:s3:::<bucket-name>/*"
      ]
    },
    {
      "Effect": "Allow",
      "Action": "cloudfront:CreateInvalidation",
      "Resource": "arn:aws:cloudfront::<account-id>:distribution/<distribution-id>"
    }
  ]
}
```

5.4. GitHub Actions Workflow

The workflow (.github/workflows/deploy.yml) triggers on pushes to the default branch, requests a short-lived OIDC token via the "id-token: write" permission, exchanges it for temporary AWS credentials scoped to the deployment role, syncs the repository contents to S3, and invalidates the CloudFront cache — see Figure 10 and Section 4.5 for the full file and a successful run.

5.5. CloudWatch Alarm & SNS Configuration

- SNS topic: website-alerts (Standard type), with a confirmed email subscription

- CloudWatch alarm metric: AWS/CloudFront → 5xxErrorRate, Statistic: Average
- Period: 5 minutes; Threshold: greater than 1%; Datapoints to alarm: 1 out of 1
- Notification action: publish to the website-alerts SNS topic on the "In alarm" state transition

A 4xxErrorRate alarm was deliberately not configured for continuous monitoring. On a low-traffic static site, a single mistyped URL can spike the 4xx rate to 100% for an evaluation window, producing noisy, low-value alerts. The team judged 5xx server-side errors — which indicate an actual infrastructure fault rather than visitor typing mistakes — to be the more meaningful signal for this project's scale.

5.6. Cost & Billing Safeguards

Because this project was intentionally kept within AWS's free usage tiers, the following safeguards were put in place during development:

- CloudFront distribution confirmed on the \$0/month Free flat-rate plan before and after creation
- No custom domain purchased — avoided recurring Route 53 / registrar charges entirely
- A zero-spend AWS Budgets alert configured to email the team immediately if any charge is incurred
- SNS usage kept well within its permanent free tier (1,000 free email deliveries per month)
- All resources planned for teardown (Section 8) immediately after documentation was captured

6. RESULTS

6.1. Secure, Private Origin

The S3 bucket remains fully private at all times — confirmed via Block Public Access settings and the auto-generated bucket policy (Figure 6), which restricts read access to exactly one CloudFront distribution ARN and no other principal.

6.2. HTTPS Delivery with No Custom Domain Cost

The site loads correctly over HTTPS via the CloudFront default domain, using the free certificate CloudFront automatically issues for its *.cloudfront.net domains — with zero certificate management overhead and no recurring domain registration cost.

6.3. End-to-End Automated Deployment

A push to the repository's main branch reliably triggers a full deployment: OIDC authentication, S3 sync, and CloudFront invalidation all complete successfully within roughly 15 seconds of workflow start (Figure 10), with changes visible on the live site shortly after.

6.4. Zero Stored Credentials

Because authentication uses IAM OIDC federation rather than access keys, no AWS credentials of any kind are stored in the GitHub repository or its secrets — removing an entire class of credential-leak risk common in simpler CI/CD setups.

6.5. Verified Alerting Pipeline

The CloudWatch alarm and SNS integration was verified using a manual state-transition test, which produced a real email delivered within roughly one minute (Figure 14). This confirms the complete alerting chain — metric evaluation, alarm state transition, and SNS delivery — functions correctly, without requiring the team to induce a genuine outage on the live site.

6.6. Cost Outcome

The entire project ran at \$0 actual cost, confirmed via the CloudFront Free plan billing indicator and AWS Billing's month-to-date charges, well within always-free service tiers for the traffic volumes involved in development, review, and alarm testing.

6.7. Demonstration Materials

Since the live AWS resources were torn down after review to avoid any ongoing cost exposure, the following materials serve as the permanent record of the working system:

- GitHub repository (source code and workflow file): <https://github.com/mohansai2976/aws-project>
- Configuration and alert screenshots included throughout Sections 4 and 6 of this report
- A live demonstration during project review, including the CloudWatch alarm test and resulting SNS email

7. TEAM CONTRIBUTIONS

This project was completed collaboratively by a team of five students from ANITS (Batch 1), under the guidance of the project mentors at APSSDC.

Name	ID Number	Institution	Focus Area
Bandi Mohan Sai Manikanta	A24126552009	ANITS	S3, CloudFront & CI/CD
Devarapalli Charan	A24126552018	ANITS	IAM & OIDC Federation
Adimolu V V Naga Durgesh	A24126552003	ANITS	CloudWatch & SNS Alerting
Sadapu Sanjay Rishi	A24126552052	ANITS	Frontend & Documentation
Tanni Dhanunjay	A24126552059	ANITS	Testing & Cost Governance

8. CHALLENGES & LEARNINGS

Several practical challenges were encountered during the build, each of which produced a useful learning outcome for the team.

8.1. Locating the "Create policy" option in IAM

While attaching a custom least-privilege policy during role creation, the console's permissions list defaulted to a search across 1,175 AWS-managed policies, and the "Create policy" action was not immediately visible. The team resolved this by creating the role first with no permissions attached, then adding an inline policy directly from the role's Permissions tab — a simpler path than navigating a separate policy-creation tab.

8.2. Static Website Hosting vs. the REST API endpoint

Early in the build, the team considered enabling S3's Static Website Hosting feature, before recognising that it requires a public bucket and is incompatible with Origin Access Control. The project instead uses S3's private REST API endpoint as the CloudFront origin, keeping the bucket fully private while CloudFront handles all public-facing delivery.

8.3. Default root object omitted on first distribution creation

The first CloudFront distribution was created without a default root object configured, meaning the root URL did not automatically resolve to index.html. This was caught during verification and corrected via the distribution's General settings, after which the site loaded correctly at the bare domain.

8.4. CloudWatch "Insufficient data" state

Immediately after creating the CloudWatch alarm, its status displayed "Insufficient data" rather than "OK" or "ALARM." The team learned this is expected behaviour for a low-traffic metric — CloudWatch simply had not yet received enough CloudFront data points to evaluate the condition — and does not indicate a misconfiguration. The alarm's manual test feature was used to verify the notification path without waiting for organic traffic.

8.5. Billing dashboard delay

The AWS Billing console displayed a message indicating a 24-hour delay before cost data would be available, which initially raised concern about undetected charges. The team confirmed this is standard AWS behaviour unrelated to actual spend, and additionally configured a zero-spend AWS Budgets alert as a faster, proactive safeguard against unexpected charges.

8.6. Choosing 5xx over 4xx for continuous alerting

The team initially considered alerting on both 4xx and 5xx error rates. After analysis, 4xx alerting was excluded from continuous monitoring, since normal visitor behaviour (mistyped URLs, stale bookmarks) can trigger frequent, low-value 4xx alerts on a low-traffic site. 5xx errors — which indicate a genuine infrastructure fault — were judged the more meaningful signal, and are the only continuously monitored alarm in the final design.

9. RESOURCE TEARDOWN

To avoid any risk of ongoing charges after project review, all AWS resources created for this project were deleted in the following order:

- CloudFront distribution disabled, then deleted once disabled
- CloudWatch alarm and SNS topic (and its email subscription) deleted
- S3 bucket emptied and deleted
- IAM role and inline policy deleted
- IAM OIDC identity provider removed (if not reused by other projects)
- AWS Billing confirmed at \$0.00 month-to-date following teardown

Because the project used only free-tier and \$0/month Free-plan services throughout, teardown was a precaution rather than a response to any incurred cost.

10. CONCLUSION

This project successfully demonstrates a secure, automated, monitored, and cost-conscious approach to static website hosting on AWS. By combining a fully private S3 origin with CloudFront's CDN and free HTTPS, the architecture avoids the common pitfall of exposing storage buckets to the public internet while still delivering fast, globally cached content.

The integration of GitHub Actions via IAM OIDC federation elevates the project beyond a one-time manual deployment into a genuine CI/CD pipeline — the same authentication pattern used in production engineering teams, without the operational overhead of storing or rotating access keys.

The addition of a CloudWatch alarm backed by Amazon SNS turns passive monitoring into active alerting, closing the loop between detecting a problem and knowing about it. Verifying this pipeline through a controlled manual test, rather than an unsafe live outage, reflects a realistic and responsible operational practice.

Equally important was the project's discipline around cost: every architectural decision, from selecting CloudFront's Free plan to omitting a purchased domain to configuring a zero-spend budget alert, was made to keep the build genuinely free while still being fully representative of a real-world deployment pattern. This makes the project a practical, low-risk template for hosting personal portfolios, documentation sites, or small production frontends.

Future enhancements could include reintroducing a custom domain via Route 53 and AWS Certificate Manager, extending alerting to cover additional CloudFront metrics, or splitting the pipeline to support separate staging and production environments through branch-based deployments.

REFERENCES

1. Amazon Web Services — Amazon S3 Documentation. <https://docs.aws.amazon.com/s3/>
2. Amazon Web Services — Amazon CloudFront Developer Guide.
<https://docs.aws.amazon.com/cloudfront/>
3. Amazon Web Services — IAM and OpenID Connect Federation.
https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_providers_oidc.html
4. Amazon Web Services — Amazon CloudWatch Alarms.
<https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/AlarmThatSendsEmail.html>
5. Amazon Web Services — Amazon SNS Developer Guide. <https://docs.aws.amazon.com/sns/>
6. GitHub — Configuring OpenID Connect in Amazon Web Services.
<https://docs.github.com/actions/deployment/security-hardening-your-deployments/configuring-openid-connect-in-amazon-web-services>
7. GitHub Actions Documentation. <https://docs.github.com/actions>